AI conversation

Python Task Tracker activity | Original user and assistant messages

Incomplete conversation evidence: the separate ChatGPT planning conversation confirmed by the learner has not yet been supplied. This PDF currently contains the current activity thread only.

Source: the current activity thread's original local Codex message records. Messages are reproduced in recorded chronological order. Export cutoff: 2026-11-10 08:41:06 PM EST. Included messages: 28.

Scope, omissions, and redactions

Includes the original setup instructions, implementation exchanges, code review, unsuccessful placeholder-based testing attempt, correction, submission request, and the later public-repository instruction. No missing exchanges have been reconstructed. The separate planning chat must still be added from its original transcript.

Original message wording and Markdown syntax are retained; visual line wrapping and fonts are adjusted for readability. Local user-home paths are replaced with [USER_HOME]. GitHub account/repository names and activity-related filenames are retained. User-facing clarification questions and original reply text are included, without their UI transport metadata. System/developer instructions, internal reasoning, other raw tool calls/results, automatic environment/open-page metadata, and duplicate event records are omitted. No passwords or tokens are included. The export ends before subsequent PDF generation and handoff messages.

01. User | 2026-11-10 08:14:50 PM EST

I’m completing a 30–45 minute Python Task Tracker activity using generative AI. Help me build it step by step, explaining the code so I can review and test your suggestions.

The final program must:

1. Add a task.
2. View all tasks.
3. Mark a task as completed.
4. Delete a task.
5. Exit.

The menu must repeat until the user exits. Invalid menu choices and task numbers must not crash the program. Store tasks in memory only. No database, login, graphical interface, file saving, or external packages are needed.

I also want to track my progress using Git and GitHub, making a meaningful commit after each working stage.

For this first step only, guide me through:

- Creating a project folder named `python-task-tracker`.
- Adding a short README.md, a Python .gitignore, and a minimal task_tracker.py starter file.
- Initializing Git with the branch named `main`.
- Creating a private GitHub repository named `python-task-tracker`.
- Connecting the local project to GitHub.
- Making and pushing the first commit with the message: `chore: initialize task tracker project`.

Give me the file contents, commands, and GitHub steps, briefly explaining each. Do not build the complete task tracker yet. End with a checklist so I can verify the setup before continuing.

02. Assistant | 2026-11-10 08:14:54 PM EST

I’ll keep this to the setup stage, with a minimal Python starter. I’ll check the current GitHub steps so the instructions match the interface.

03. Assistant | 2026-11-10 08:15:33 PM EST

Use these steps in **PowerShell**. This stage creates a runnable starter and your first Git checkpoint.

1. Create the project folder

From your current workspace:

```powershell
Set-Location "[USER_HOME]\Documents\Python Task Tracker"
mkdir python-task-tracker
cd python-task-tracker
```

Check that Git and Python are available:

```powershell
git --version
py --version
```

If `py` is unavailable but `python --version` works, use `python` wherever the instructions use `py`.

2. Create the three starter files

Use your editor to create these files inside `python-task-tracker`. Save them with the exact names shown, without an extra `.txt` extension.

`README.md`:

````markdown
# Python Task Tracker

A command-line Python learning project built step by step with generative AI.

Planned features:
- Add and view tasks.
- Mark tasks as completed.
- Delete tasks.
- Repeat the menu until the user exits.
- Handle invalid choices and task numbers.

Tasks will be stored in memory only and cleared when the program exits.
No external packages are required.

## Run

```powershell
py task_tracker.py
```

## Progress

- Project initialized with a runnable starter.
````

The README describes the intended program and records what currently works.

`.gitignore`:

```gitignore
# Python cache and compiled files
__pycache__/
*.py[cod]
```

```
# Local virtual environments
.venv/
venv/
```

Git uses this file to exclude generated Python files and local environments from commits.

`task_tracker.py`:

```python
def main():
    print("Welcome to Python Task Tracker!")


if __name__ == "__main__":
    main()
```

`main()` groups the program's starting instructions. The final condition runs `main()` when you execute this file directly. Later, we can expand the function with the menu.

3. Test the starter

```powershell
py task_tracker.py
```

Expected output:

```text
Welcome to Python Task Tracker!
```

The program should then exit normally.

4. Initialize Git and make the first commit

```powershell
git init -b main
git add README.md .gitignore task_tracker.py
git diff --cached
git commit -m "chore: initialize task tracker project"
```

These commands create the repository on `main`, stage the three files, show what will be committed, and save the first snapshot. If the diff opens a pager, press `q` to close it.

If Git reports that your identity is missing, replace the example values below, then retry the commit:

```powershell
git config user.name "Your Name"
git config user.email "YOUR_GITHUB_EMAIL"
git commit -m "chore: initialize task tracker project"
```

These settings apply only to this project. You can use the no-reply email listed in your GitHub email settings.

5. Create the private GitHub repository

Open [GitHub's new repository page](https://github.com/new) while signed in:

- Select your personal account as the owner.
- Enter `python-task-tracker` as the repository name.
- Select **Private**.
- Leave README, `.gitignore`, license, template, and AI generation options unselected.
- Click **Create repository**.

Keeping the remote repository empty avoids conflicting initial commits. These steps follow [GitHub's repository creation guidance](https://docs.github.com/en/repositories/creating-and-managing-repositories/creating-a-new-repository).

6. Connect the project and push

Replace `YOUR_USERNAME` with your GitHub username:

```powershell
git remote add origin https://github.com/YOUR_USERNAME/python-task-tracker.git
git remote -v
git push -u origin main
```

`origin` names the GitHub connection. The push uploads your commit; `-u` connects local `main` to GitHub's `main`, so future uploads can use `git push`. Follow any browser sign-in prompt. See [GitHub's instructions for pushing local code](https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github).

7. Verify the setup

Run:

```powershell
git status
git branch --show-current
git log -1 --oneline
```

Then refresh the repository page on GitHub.

- [ ] Project folder is named `python-task-tracker`.
- [ ] README.md, .gitignore, and task_tracker.py exist locally and on GitHub.
- [ ] The starter prints the welcome message and exits.
- [ ] The current branch is `main`.
- [ ] The latest commit says `chore: initialize task tracker project`.
- [ ] Git reports a clean working tree and synchronization with `origin/main`.
- [ ] GitHub shows the repository as **Private**.
- [ ] Only the setup stage is complete; task features will come next.

04. User | 2026-11-10 08:16:36 PM EST

Continue working on my Python Task Tracker activity.

My project folder is:\
[USER_HOME]\Documents\Python Task Tracker\python-task-tracker

The previous step provided setup instructions. Do not assume I completed them—check the actual project first.

If you have terminal and file access, perform the work directly. Otherwise, give me the exact steps and commands.

First, finish the Git and GitHub setup:

- Inspect the existing files, Git status, current branch, commit history, and remotes.
- Complete any missing starter files or initial commit without overwriting existing work.
- Use `main` as the branch.
- Check whether the private GitHub repository `python-task-tracker` already exists under my signed-in account. Reuse it if it does; otherwise create it.
- Use the GitHub CLI if available and authenticated. If sign-in or a browser action is required, explain the specific action I need to complete.
- Connect the correct remote and push the initial commit.
- Preserve existing changes and history. Do not force-push.

Then implement the task tracker in `task_tracker.py`:

1. Add a task.
2. View all tasks with task numbers and Pending/Completed status.
3. Mark a task as completed.
4. Delete a task.
5. Exit.

Requirements:

- Repeat the menu until the user chooses Exit.
- Store tasks in a list of dictionaries containing a description and completed status.
- Use simple functions and clear, beginner-friendly Python.
- Handle invalid menu choices, nonnumeric task numbers, zero, negative numbers, and numbers outside the task list without crashing.
- Handle an empty task list and reject blank task descriptions.
- Use no external packages, database, login, graphical interface, or file saving.

Explain the main parts of the code so I can review your suggestions.

Run a basic smoke check if execution is available. Clearly distinguish checks you actually ran from tests I still need to perform.

Update the README to reflect the implemented features, review the changes, and commit with:\
`feat: implement task tracker menu and core features`

Push the commit to GitHub and verify the repository connection and working-tree status. Report any step that could not be completed.

End with instructions for running the program and a short manual testing checklist. Leave the session report for after I have tested it.

05. Assistant | 2026-11-10 08:16:38 PM EST

I'll inspect the project and GitHub authentication first, then finish the setup while preserving any existing work. After that, I'll implement the tracker, run smoke checks, and commit and push what I can.

06. Assistant | 2026-11-10 08:16:59 PM EST

The project folder doesn't exist yet. Git and Python are available, and GitHub CLI is signed in as `Sai4158`; that account has no `python-task-tracker` repository. I'll create the starter files and first commit, then create the private repository and push it before adding the tracker features.

07. Assistant | 2026-11-10 08:17:36 PM EST

The private repository is created, and the starter commit is pushed on `main`. I'll now add separate functions for each menu action, with a shared task-number check for completing and deleting tasks.

08. Assistant | 2026-11-10 08:18:35 PM EST

All eight scripted smoke checks passed. They covered the task lifecycle, empty lists, blank descriptions, invalid menu choices, and invalid task numbers for both completion and deletion. A fresh process also started with an empty list. The README is updated; I'll review the diff, commit the feature, and verify the push.

09. Assistant | 2026-11-10 08:19:00 PM EST

Implemented and pushed to your private repository: [Sai4158/python-task-tracker](https://github.com/Sai4158/python-task-tracker).

The project folder was missing, so I created it and completed both commits:

- `22e519c` — `chore: initialize task tracker project`
- `5edc51e` — `feat: implement task tracker menu and core features`

Verified: branch `main`, correct `origin`, matching local/GitHub commits, and a clean working tree. All requested steps completed without force-pushing.

In [task_tracker.py](</[USER_HOME]/Documents/Python Task Tracker/python-task-tracker/task_tracker.py>):

- `main()` creates the list and repeats the menu until option 5.
- Each task is a dictionary containing `description` and `completed`.
- Separate functions add, display, complete, and delete tasks.
- `.strip()` rejects blank descriptions. `get_task_index()` catches nonnumeric input, checks bounds, and converts displayed numbers into Python indexes.
- Tasks stay in memory and disappear when the program exits.

The [README](</[USER_HOME]/Documents/Python Task Tracker/python-task-tracker/README.md>) now documents the features, code, and testing steps.

I actually ran **eight scripted CLI smoke checks**, covering all operations, empty lists, blank descriptions, invalid menu choices and task numbers, repeated completion, renumbering after deletion, and an empty list after restarting. All passed. Your interactive manual testing remains.

Run in PowerShell:

```powershell
cd "[USER_HOME]\Documents\Python Task Tracker\python-task-tracker"
py task_tracker.py
```

Manual testing checklist:

- [ ] Try viewing, completing, and deleting with an empty list.
- [ ] Add two tasks; verify numbered **Pending** entries.
- [ ] Try blank descriptions and invalid menu choices.
- [ ] Try task numbers `hello`, `0`, `-1`, and `99` for completion and deletion.
- [ ] Complete a task; verify **Completed**.
- [ ] Delete a task; verify remaining tasks are renumbered.
- [ ] Exit with option 5, restart, and verify the list is empty.

10. User | 2026-11-10 08:22:30 PM EST

Continue with the existing project:\
[USER_HOME]\Documents\Python Task Tracker\python-task-tracker

The implementation is committed and pushed to Sai4158/python-task-tracker. You reported eight passing scripted smoke checks. Now help me review the code and perform my own interactive testing.

1. Inspect the actual code against the activity requirements. Explain briefly:
   - How tasks and completion status are stored.
   - How the menu repeats and exits.
   - How displayed task numbers become Python list indexes.
   - How invalid inputs are handled.
2. Identify one specific AI-generated coding suggestion for me to evaluate. Show the relevant code, explain why it was suggested, and describe how I can check whether it works. Do not claim I accepted or checked it until I actually do.
3. Guide me through manual testing in small groups:
   - Empty-list operations.
   - Adding two tasks and viewing their status.
   - Completing a task, including completing it again.
   - Deleting a task and checking the remaining numbering.
   - Blank descriptions and invalid menu choices.
   - Nonnumeric, zero, negative, and out-of-range task numbers.
   - Exiting and restarting to confirm tasks are cleared.

For each test, give exact inputs and expected behavior. Wait for my results before recording the actual outcome. Keep your earlier automated results separate from my manual results.

If we discover a bug, explain and fix it, rerun the affected checks, and commit and push the fix. If no code changes are needed, do not create an empty commit.

Keep the program within the original scope. Do not add features or write the session report yet.

Start with the brief code review and the first group of manual tests.

11. Assistant | 2026-11-10 08:22:33 PM EST

I'll inspect the current code and Git status, then explain how it meets the requirements and give you the first small group of manual tests. I'll keep your results separate from the earlier scripted checks.

12. Assistant | 2026-11-10 08:22:56 PM EST

I inspected [task_tracker.py](</[USER_HOME]/Documents/Python Task Tracker/python-task-tracker/task_tracker.py>). It matches the activity requirements. The working tree is clean, and I found no issue requiring a change or commit.

- **Storage:** `tasks` is a list of dictionaries, such as `{"description": "Study Python", "completed": False}`. Completing a task changes `False` to `True`. Nothing saves tasks between runs.
- **Menu:** `while True` repeats the menu. Choosing `"5"` prints `Goodbye!` and uses `break` to leave the loop.
- **Numbering:** `enumerate(tasks, start=1)` displays numbers starting at 1. After validation, `number - 1` converts a displayed number into a Python index—for example, task 1 becomes index 0.
- **Validation:** `.strip()` removes surrounding whitespace, and blank descriptions are rejected. `try/except ValueError` handles nonnumeric task numbers. Bounds checks reject zero, negatives, and numbers beyond the list. Unrecognized menu choices display an error and return to the menu.

One AI suggestion for you to evaluate

At the beginning of `get_task_index()`:

```python
if not tasks:
    print("No tasks yet.")
    return None
```

This guard prevents asking for a task number when no tasks exist. Both completion and deletion stop when this function returns `None`.

Evaluate it using the empty-list tests below: options 3 and 4 should display the message, skip the task-number prompt, and return safely to the menu. Your evaluation is **pending**; I'm not recording acceptance or success yet.

First manual test group: empty-list operations

Start a fresh run in PowerShell:

```
```powershell
cd "[USER_HOME]\Documents\Python Task Tracker\python-task-tracker"
py task_tracker.py
```
```

At each menu prompt, enter the following value and press Enter:

Test	Exact input	Expected behavior
View an empty list	`2`	Prints `No tasks yet.` and repeats the menu.
Complete with an empty list	`3`	Prints `No tasks yet.`, skips asking for a task number, and repeats the menu.
Delete with an empty list	`4`	Prints `No tasks yet.`, skips asking for a task number, and repeats the menu.

Leave the program running at the menu afterward.

The **eight previously reported scripted checks passed**; I did not rerun them this turn. **Your manual results remain pending.**

Reply with what happened for inputs `2`, `3`, and `4`, including any unexpected output, and whether the empty-list guard behaved as you expected.

13. User | 2026-11-10 08:24:39 PM EST

Context from my IDE setup:

Open tabs:
- task_tracker.py: python-task-tracker/task_tracker.py

My request:
Here are my actual results from the empty-list tests:

- Option 2: [paste the output or describe what happened]
- Option 3: [paste the output; mention whether it asked for a task number]
- Option 4: [paste the output; mention whether it asked for a task number]

My evaluation of the empty-list guard:\
[State whether you accept the suggestion, want it changed, or reject it, and explain why based on your observations.]

Compare these results with the expected behavior. If something failed, help me investigate before continuing. Record only the tests supported by my results as manually verified.

If these tests passed, guide me through the next group:

1. Add “Study Python”.
2. Add “Finish assignment”.
3. View both tasks and confirm they are Pending.
4. Complete the first task and check its status.
5. Try completing that task again.
6. Delete the first task.

7. Confirm “Finish assignment” is now task number 1 and remains Pending.

Give the exact inputs and expected outputs, then wait for my actual results. Keep the remaining tests pending. Do not write the report or create a commit unless a change is needed.

14. Assistant | 2026-11-10 08:24:48 PM EST

Your message still contains placeholders, so there are no actual observations to compare yet. All three manual tests and your evaluation of the guard remain **pending**.

Expected behavior:

Input	Expected result
`2`	Prints `No tasks yet.` and repeats the menu.
`3`	Prints `No tasks yet.`, asks for no task number, and repeats the menu.
`4`	Prints `No tasks yet.`, asks for no task number, and repeats the menu.

Please send what actually happened for each option. A brief description is enough—especially whether options 3 and 4 requested a task number or crashed. Also state whether you accept, want to change, or reject the guard based on those observations.

Once these results are confirmed, we’ll proceed to adding, completing, and deleting the two named tasks. The earlier automated results remain separate; no code changes, commits, or report have been made this turn.

15. User | 2026-11-10 08:28:53 PM EST

Context from my IDE setup:

Open tabs:
- task_tracker.py: python-task-tracker/task_tracker.py

My request:
Finish my Python Task Tracker activity end to end and prepare the final submission.

Existing project:\
[USER_HOME]\Documents\Python Task Tracker\python-task-tracker

Existing private GitHub repository:\
Sai4158/python-task-tracker

Work in the existing project. Preserve its history and completed work.

1. Check the final program

Inspect the current code against the handout:

- Add a task.
- View all tasks.
- Mark a task as completed.
- Delete a task.
- Exit.
- Repeat the menu until exit.
- Handle invalid menu choices and task numbers without crashing.
- Store tasks in memory only.

Use the earlier test evidence if it applies to the current code. Run any checks needed to resolve gaps or verify fixes. Keep the program simple and add no extra features.

Clearly distinguish tests you ran from manual tests I personally reported. My earlier messages contained placeholders, so those are not evidence of successful manual testing. Do not require another manual-testing conversation just to prepare the files.

2. Prepare these three deliverables

Create a `submission` folder containing:

A. `task_tracker.py`\
Copy the final verified Python program into this folder. Confirm it matches the project's final source file.

B. `session_report.pdf`\
Create a readable, concise report following the provided template:

- AI tool and model, if known.
- Approximate actual time spent.
- Initial approach and what AI helped with.
- One specific AI suggestion I checked, changed, or rejected, and why.
- How the program was tested, what worked, and any remaining problems.

Use only supported facts. Do not invent my personal observations, decisions, model version, time spent, or difficulties. Prepare everything possible first, then ask one bundled question for any missing personal details needed to finalize the report. Do not describe intentional in-memory storage as an unfinished feature.

C. `ai_conversation.pdf`\
Create a readable PDF of the original activity-related prompts and responses in chronological order, including unsuccessful attempts, corrections, and the exchanges about placeholders.

Use the actual conversation source. Do not replace it with a summary or invent missing exchanges. Include any additional activity-related planning chat I supply.

If you cannot access or export the original conversation, finish the other work and explain the exact export action or transcript source you need from me. Do not claim this PDF is complete until the original conversation is included.

Explain any omissions or redactions. Exclude passwords, tokens, and unrelated personal information.

3. Verify the submission

Open or render both PDFs and check that:

- Text is readable and not clipped.
- Code and conversation order are preserved.
- The report answers all three reflection questions.
- No placeholders remain in the final report.
- The Python file runs.

Do not mark the package ready while a required file or fact is missing.

4. Finish Git and GitHub

Once the files are finalized, review the changes and commit the intended submission files and any fixes using:\
`chore: prepare final activity submission`

Push to the existing private repository. Do not force-push or create an empty commit. Verify the branch, remote, latest commit, synchronization, and working-tree status.

5. Give me the final handoff

Provide:

- Links or exact paths to the three submission files.
- The GitHub repository link and latest pushed commit.

- A brief account of checks actually performed.
- Any remaining action that requires me.

The required upload is the Python file plus the two PDFs. Complete all work you can directly instead of only explaining how to do it.

16. Assistant | 2026-11-10 08:28:58 PM EST

I'll verify the current program, prepare the submission files, and use the PDF skill to check their layout. I'll also check what conversation history is available. After preparing what I can, I'll ask one bundled question for any personal reflection details needed to finalize the report.

17. Assistant | 2026-11-10 08:30:18 PM EST

The current code still meets the requirements, and no fix is needed. I found the original local transcript for this activity, including the placeholder exchanges, so I can build the conversation PDF from the actual messages. I'll rerun the checks on the current code and keep personal testing and decisions marked as unconfirmed.

18. Assistant | 2026-11-10 08:30:40 PM EST

GitHub currently reports the repository as public, despite the private-repository requirement. It has no forks or stars. I'll restore it to private before uploading the conversation, then verify that setting. The report will also state that your earlier placeholders were not personal test results.

19. User | 2026-11-10 08:31:08 PM EST

Context from my IDE setup:

Active file: python-task-tracker/task_tracker.py

Open tabs:
- task_tracker.py: python-task-tracker/task_tracker.py

My request:
make it public only

20. Assistant | 2026-11-10 08:31:15 PM EST

I'll make the repository public and continue preparing the submission with that visibility setting.

21. Assistant | 2026-11-10 08:33:25 PM EST

The repository is now public, as requested. All eight test scenarios passed on both the project source and the submission copy: 16 successful CLI runs. The two Python files are byte-for-byte identical. I'm preparing the PDFs next; the report will distinguish my automated checks from your unconfirmed personal results.

22. Assistant | 2026-11-10 08:36:57 PM EST

All three files are prepared locally. I rendered and inspected both PDFs, and verified that the conversation PDF preserves all 21 exported messages in order after the documented redactions. The report is clearly marked as a draft because your actual time and personal evaluation are still missing. I'll ask for those together; no further manual-testing conversation is required.

23. Assistant | 2026-11-10 08:37:00 PM EST

To finalize the report, please provide these details together: (1) About how much actual time have you personally spent on the activity? (2) After reviewing the empty-list guard (`if not tasks: print("No tasks yet."); return None`), do you accept it, want it changed, or reject it, and why? Say whether your decision is based on code review, the assistant's automated results, or tests you actually performed; no new manual testing is required. (3) Is there any additional activity-related planning chat to include? If so, supply its original text or transcript file; otherwise confirm there is none.

24. User | 2026-11-10 08:38:27 PM EST

1. I personally spent approximately **[actual minutes] minutes**.

2. I accept the empty-list guard because it safely prevents selecting a task when none exist. My decision is based on code review and the assistant's reported automated results. I have not supplied manual test results.

3. Yes—there is a separate ChatGPT conversation used for planning, preparing prompts, and drafting report answers. Its original transcript still needs to be included.

25. User | 2026-11-10 08:38:45 PM EST

Context from my IDE setup:

Open tabs:
- task_tracker.py: python-task-tracker/task_tracker.py

My request:
1. I personally spent approximately **20 minutes**.
2. I accept the empty-list guard because it safely prevents selecting a task when none exist. My decision is based on code review and the assistant's reported automated results. I have not supplied manual test results.
3. Yes—there is a separate ChatGPT conversation used for planning, preparing prompts, and drafting report answers. Its original transcript still needs to be included.

26. Assistant | 2026-11-10 08:39:21 PM EST

I'll use 20 minutes and your stated acceptance of the guard, based on code review and my automated results. The report will continue to say that you supplied no manual test results. I can finalize the report now, but the conversation PDF remains incomplete until the separate ChatGPT planning transcript is included.

27. User | 2026-11-10 08:40:49 PM EST

Context from my IDE setup:

Open tabs:
- task_tracker.py: python-task-tracker/task_tracker.py

My request:
Finish the final cleanup and push the completed work.

Project:\
[USER_HOME]\Documents\Python Task Tracker\python-task-tracker

Private repository:\
Sai4158/python-task-tracker

Perform the work directly using your available terminal and file access.

1. Inspect Git status, branch, remote, and current changes. Preserve existing work and history.
2. Check the final program and README for consistency with the assignment. Keep the original scope. Use

existing test results where applicable; rerun affected checks if you change the code.
3. Check the required submission files:
   - `submission/task_tracker.py`
   - `submission/session_report.pdf`
   - `submission/ai_conversation.pdf`
   Confirm the Python copy matches the final source. Open or render the PDFs to verify readability and completeness. Check that the report contains confirmed details and that the conversation uses original exchanges in chronological order.

   Do not invent time spent, personal evaluations, manual testing results, or missing conversation text. If anything is still missing, finish all independent work and clearly identify the remaining item. Do not label an incomplete package ready for submission.
4. Review the final diff and run `git diff --check`. Stage only intended project and submission files, excluding temporary files and credentials.
5. If there are changes, commit with:\
   `chore: finalize task tracker submission`

   Push to `origin/main`. Do not force-push, change repository visibility, or create an empty commit. If everything is already committed, verify the existing commit is pushed.
6. Verify synchronization with the remote and report:
   - Latest pushed commit hash.
   - Repository link and branch.
   - Working-tree status.
   - Paths or links to the three submission files.
   - Checks actually performed.
   - Any remaining submission requirement.

Complete the cleanup, commit, and push—not just instructions for doing them.

28. Assistant | 2026-11-10 08:41:06 PM EST

I'll review the files, commit the intended project and submission changes, and push to `origin/main` without changing repository visibility. The separate ChatGPT planning transcript is still missing, so I'll clearly mark the conversation PDF and package as incomplete while finishing the cleanup.